Bonus - A

徐晟洋 3230104220

1 Task1

1.1 管理文件对象的生命周期

首先得在内核里建立起“文件”和“文件描述符”的概念。在 fs.c 里，我实现了一系列基础的 plumbing 代码（比如分配、查找、释放），主要是为了维护进程结构体里的 files_struct。

核心是 file_init 函数，除了初始化缓存，最重要的就是硬编码创建 fd=0, 1, 2 这三个文件对象，分别对应 stdin, stdout, stderr，并把它们挂到进程的文件链表上。

代码片段 (fs.c):

void file_init(struct files_struct *files)
{
  // ... 初始化链表 ...

```
// 手动创建 stdin (fd=0)
struct file *stdin = kmem_cache_alloc(file_cache);
stdin->fd = 0;
stdin->opened = 1;
stdin->read = stdin_read; // 绑定读函数
list_add_tail(&stdin->list, &files->fd_list);

// 手动创建 stdout (fd=1)
struct file *stdout = kmem_cache_alloc(file_cache);
stdout->fd = 1;
stdout->opened = 1;
stdout->write = stdout_write; // 绑定写函数
list_add_tail(&stdout->list, &files->fd_list);

// ... stderr 同理 ...
```

}

1.2 实现 VFS 层的读写接口

有了文件对象，还得有函数去真正干活。这部分是在 vfs.c 里实现的。

这里利用了 C 语言的多态思想：struct file 里有函数指针。对于标准输入输出，底层其实就是调用 SBI 的接口（sbi_console_putchar 和 sbi_console_getchar）来和 UART 交互。

代码片段 (vfs.c):

```
// 标准输出写函数
int64_t stdout_write(struct file *file, const void *buf, uint64_t len)
{
    const char *ptr = buf;
    // 简单粗暴，直接通过 SBI 一个字符一个字符打印出去
```

```
    for (uint64_t i = 0; i < len; i++) {
        sbi_console_putchar(ptr[i]);
    }
    return len;
}

// 标准输入读函数
int64_t stdin_read(struct file *file, void *buf, uint64_t len)
{
    char *ptr = buf;
    for (uint64_t i = 0; i < len; i++) {
        // 调用 SBI 获取字符，这里是阻塞的
        int c = sbi_console_getchar();
        if (c < 0) break; // 出错了
        ptr[i] = (char)c;
    }
    return i;
}
```

1.3 改造 System Call

最后一步就是把用户态的 write 和 read 系统调用对接上来。

在 syscall.c 里，我修改了 sys_write 和 sys_read。逻辑很简单：拿着用户给的 fd，去当前进程的文件表里查对应的 struct file ，查到了就调用它绑定的函数指针。

代码片段 (syscall.c):

```
int64_t sys_write(unsigned int fd, const char *buf, size_t count)
{
    // 1. 查找文件对象
    struct file *f = find_file_by_fd(&current->files, fd);
    if (!f) return -1;
  // 2. 检查权限和函数指针
if ((f->perms & FILE_WRITABLE) && f->write) {
    // 3. 调用具体实现的写函数（即上面的 stdout_write）
    return f->write(f, buf, count);
}
return -1;

}
```

1.4 验证结果

```
root@zju-os /New_OS (bonus-a)# make run
SBI machine vendor id: 0
SBI machine arch id: 0
SBI machine imp id: 0
[Lab1] sstatus: 0x8000000200006000
[Lab1] sie: 00000000
[Lab1] sip: 0x000020
[Lab1] time spent for 10000 loops: 0 seconds (47 clocks)
[INFO, PID = 0] proc.c:270:kernel_thread: created [pid = 1, fn = 0xffffffd600200c9c, arg = 0, task = 0xffffffd600329128]
[Lab1] Enabling Timer Interrupts
[INFO, PID = 0] proc.c:291:user_mode_thread: user process created [pid = 2]
hello, stdout!
hello, stderr!
SHELL > echo hello
hello
SHELL > QEMU 10.1.0 monitor - type 'help' for more information
(qemu) quit
```

2 Task 2：实现 FAT32 文件系统

2.1 FAT32 的核心逻辑

1. 初始化 (fat32_init)：

读第 0 号扇区（BPB），把一些关键参数（像每簇扇区数 sec_per_clus、保留扇区数、FAT表的位置等）存到全局变量 fat32_volume 里，后面算地址全靠它。

2. 打开文件 (fat32_open_file)：

实验简化了，只用管根目录。思路就是算出根目录所在的扇区，然后暴力遍历每一个目录项 (fat32_dir_entry)。比较文件名的时候要注意，FAT32 存的名字没有 .，是 8字节文件名 + 3字节扩展名，而且要忽略大小写。

```
// 简略版逻辑
for (int i = 0; i < entries_count; i++) {
    read_sector(current_sector, buf);
    struct fat32_dir_entry *entry = (struct fat32_dir_entry *)buf + offset;
    if (match_filename(entry->name, target_name)) {
        // 找到了！记录起始簇号和文件大小
        file->fat32_file.cluster = entry->cluster;
        return 0;
    }
}
```

3. 读写 (fat32_read / fat32_write)：

这是难点。逻辑是：根据当前文件的读写指针 cfo，算出它在文件的第几个“簇”里，如果跨簇了，就要查 FAT 表找下一个簇 (next_cluster)。

写操作其实就是：读扇区 -> 修改内存里的 buffer -> 写回扇区。

```c
// 跨簇查找逻辑
while (cfo >= cluster_size) {
    cluster = next_cluster(cluster); // 查表找下一簇
    cfo -= cluster_size;
}
// 读/写扇区
uint64_t sector = cluster_to_sector(cluster) + offset_in_cluster_sec;
virtio_blk_read_sector(sector, buf);
// ... memcpy ...
if (is_write) virtio_blk_write_sector(sector, buf);
```

4. Lseek (fat32_lseek):

这个比较简单，主要就是更新 file->cfo。之前这里有个 bug 是把 cfo 暴力置 0 了，修正后要根据 SEEK_SET, SEEK_CUR, SEEK_END 来算。

2.2 中间层：文件抽象 VFS

为了让内核不管是对接标准输出（stdout）还是磁盘文件都用统一的接口，我们需要复用 struct file。

file_open:

这是个分发器。如果路径是以 /fat32/ 开头的，就调用上面的 fat32_open_file，并且把 f->read 和 f->write 指针指向 FAT32 的实现；否则如果是 stdin/stdout 就指过去。

```c
struct file *file_open(struct files_struct *files, const char *path, int flags) {
    struct file *f = alloc_file(); // 申请一个空文件结构
    if (strncmp(path, "/fat32/", 7) == 0) {
        f->fs_type = FS_TYPE_FAT32;
        f->read = fat32_read;      // 挂载函数指针
        f->write = fat32_write;
        f->lseek = fat32_lseek;
        fat32_open_file(f, path);
    }
    // ...
    return f;
}
```

2.3 系统调用与内存权限

这一步把用户态的 cat 和 edit 命令通到内核。

1. Syscalls (sys_openat, sys_close, sys_lseek):

基本就是参数透传，调用 VFS 层的函数。

2. 读写系统调用 (sys_read, sys_write) 这里被坑了

最开始实现的时候，cat 命令一直挂掉。原因是我们直接把用户态传进来的 buffer 指针 (regs->a1) 拿给内核里的 memcpy 用了。

但 RISC-V 默认内核态（S 模式）是不能直接访问用户态（U 模式）内存的（为了安全）。

解决方法：在操作前手动开启 sstatus 寄存器的 SUM (Supervisor Access User Memory) 位。

long sys_read(const struct pt_regs *regs) {
  // ... 获取 fd 和 file ...

```
// 关键：开启 SUM 位，允许内核访问用户指针 buf
asm volatile("csrr %0, sstatus" : "=r"(sstatus));
asm volatile("csrs sstatus, %0" :: "r"(SSTATUS_SUM));

long ret = f->read(f, buf, len); // 真正干活

// 恢复现场
asm volatile("csrw sstatus, %0" :: "r"(sstatus));
return ret;
```

}

2.4 验证结果

可以正常读取和修改 FAT32 文件系统中的文件内容

```
[INFO, PID = 0] proc.c:291:user_mode_thread: user process created [pid = 2]
hello, stdout!
hello, stderr!
SHELL > cat /fat32/email
From: TORVALDS@klaava.Helsinki.FI (Linus Benedict Torvalds)
Newsgroups: comp.os.minix
Subject: What would you like to see most in minix?
Summary: small poll for my new operating system
Message-ID:
Date: 25 Aug 91 20:57:08 GMT
Organization: University of Helsinki

Hello everybody out there using minix -

I'm doing a (free) operating system (just a hobby, won't be big and professional like gnu) for 386(486) AT
cones. This has been brewing since april, and is starting to get ready. I'd Tike any feedback on things peo
ple like/dislike in minix, as my 0S resembles it somewhat (same physical layout of the file-system (due to
practical reasons) among other things) .

I've currently ported bash(1.08) and gcc(1.40), and things seem to work. This implies that I'll get somethi
ng practical within a few months, andI'd like to know what features most people would want. Any suggestions
 are welcome, but I won't promise I'll implement them :-)

               Linus (torvalds@kruuna.helsinki.fi)

PS. Yes . it's free of any minix code, and it has a multi-threaded fs. It is NOT protable (uses 386 task sw
itching etc), and it probably never will support anything other than AT-hard disks, as that's all I have :-
($
```

3 实现程序加载

3.1 进程生命周期管理

1. 进程结构体：利用 struct task_struct 中的 child_exit_comp（子进程退出完成量）和 parent 指针。
2. 初始化 (task_init)：为 init_task 初始化了 child_exit_comp，确保其作为父进程时 wait 机制正常。
3. 拷贝进程 (copy_process)：在 fork 时为新进程初始化 child_exit_comp，确保每个进程都有自己的等待队列。
4. 进程退出 (do_exit)：
   (a) 设置退出码 exit_code。
   (b) 关键点：检查是否存在父进程 (current->parent)，若存在则调用 complete(&current->parent->child_exit_comp) 唤醒父进程。保留了 wait_comp 的兼容逻辑，确保内核启动时能正确等待 nish 初始化。

3.2 程序加载与系统调用

1. user_mode_thread：接口从接收内存地址 void * 修改为文件路径 const char *path。现在通过 load_elf_binary(path, ...) 直接从文件系统加载初始进程 /fat32/nish。
2. sys_wait：内核现有的 sys_wait 实现会阻塞在 current->child_exit_comp 上，这与我们在 do_exit 中添加的唤醒逻辑相匹配。
3. sys_execve：通过 file_open 打开 ELF 文件，并调用 load_elf_binary 读取文件头和程序段（采用 Eager Loading 策略，即 exec 时一次性读取所有 PT_LOAD 段到内存）。

3.3 验证结果

在 nish中执行 run命令：

- `run /fat32/print`：成功输出了测试字符串。
- `run /fat32/cow`：成功运行 Copy-on-Write 测试，父子进程正确共享和复制变量。
- `run /fat32/fib`：成功运行斐波那契数列测试。
- `run/fat32/mfork`：成功启动多进程压力测试。

```
hello, stdout!
hello, stderr!
SHELL > run /fat32/print
[INFO, PID = 2] syscall.c:143:sys_clone: fork new process [pid = 3]

[Lab4 User] PID = 3
3456789abcdef012345SHELL > run /fat32/cow
[INFO, PID = 2] syscall.c:143:sys_clone: fork new process [pid = 4]

[Lab5 CoW] Copy-On-Write Test Start
[Lab5 CoW] [T1-Pre] [PID = 4] var = 0
[Lab5 CoW] [T1-Pre] [PID = 4] var = 1
[INFO, PID = 4] syscall.c:143:sys_clone: fork new process [pid = 5]
[Lab5 CoW] [PARN] [PID = 4] message at space[0x1000]: ZJU OS 2025 Lab5 - COW Test
[Lab5 CoW] [PARN] [PID = 4] var = 2 (COW check)
[Lab5 CoW] [PARN] [PID = 4] var = 3 (COW check)
[Lab5 CoW] [CHLD] [PID = 5] message at space[0x1000]: ZJU OS 2025 Lab5 - COW Test
[Lab5 CoW] [CHLD] [PID = 5] var = 2 (COW check)
[Lab5 CoW] [CHLD] [PID = 5] var = 3 (COW check)
[Lab5 CoW] [CHLD] [PID = 5] var = 4 (COW check)
[Lab5 CoW] [CHLD] [PID = 5] var = 5 (COW check)
[Lab5 CoW] [PARN] [PID = 4] var = 4 (COW check)
[Lab5 CoW] [PARN] [PID = 4] var = 5 (COW check)
[Lab5 CoW] [PARN] [PID = 4] var = 6 (COW check)
[Lab5 CoW] [CHLD] [PID = 5] var = 6 (COW check)
```

```
SHELL > run /fat32/fib
[INFO, PID = 2] syscall.c:143:sys_clone: fork new process [pid = 6]

[Lab5 Fib] Large Array & Fib Test Start
[INFO, PID = 6] syscall.c:143:sys_clone: fork new process [pid = 7]
[Lab5 Fib] [Init] fork returns 7
[Lab5 Fib] [PARN] [PID=6] 0th fib = 1, bigarr[999]=2998
[Lab5 Fib] [PARN] [PID=6] 1st fib = 1, bigarr[983]=2950
[Lab5 Fib] [PARN] [PID=6] 2nd fib = 1, bigarr[974]=2923
[Lab5 Fib] [PARN] [PID=6] 3rd fib = 2, bigarr[964]=2893
[Lab5 Fib] [PARN] [PID=6] 4th fib = 3, bigarr[951]=2854
[Lab5 Fib] [Init] fork returns 0
[Lab5 Fib] [CHLD] [PID=7] 0th fib = 1, bigarr[999]=2998
[Lab5 Fib] [CHLD] [PID=7] 1st fib = 1, bigarr[983]=2950
[Lab5 Fib] [CHLD] [PID=7] 2nd fib = 1, bigarr[974]=2923
[Lab5 Fib] [CHLD] [PID=7] 3rd fib = 2, bigarr[964]=2893
[Lab5 Fib] [CHLD] [PID=7] 4th fib = 3, bigarr[951]=2854
[Lab5 Fib] [PARN] [PID=6] 5th fib = 5, bigarr[949]=2848
[Lab5 Fib] [PARN] [PID=6] 6th fib = 8, bigarr[935]=2806
[Lab5 Fib] [PARN] [PID=6] 7th fib = 13, bigarr[929]=2788
[Lab5 Fib] [PARN] [PID=6] 8th fib = 21, bigarr[911]=2734
[Lab5 Fib] [PARN] [PID=6] 9th fib = 34, bigarr[907]=2722
[Lab5 Fib] [CHLD] [PID=7] 5th fib = 5, bigarr[949]=2848
[Lab5 Fib] [CHLD] [PID=7] 6th fib = 8, bigarr[935]=2806
[Lab5 Fib] [CHLD] [PID=7] 7th fib = 13, bigarr[929]=2788
[Lab5 Fib] [CHLD] [PID=7] 8th fib = 21, bigarr[911]=2734
[Lab5 Fib] [CHLD] [PID=7] 9th fib = 34, bigarr[907]=2722
[Lab5 Fib] [PARN] [PID=6] 10th fib = 55, bigarr[890]=2671
[Lab5 Fib] [CHLD] [PID=7] 10th fib = 55, bigarr[890]=2671
[Lab5 Fib] [PARN] [PID=6] 15th fib = 610, bigarr[840]=2521
[Lab5 Fib] [CHLD] [PID=7] 15th fib = 610, bigarr[840]=2521
[Lab5 Fib] [PARN] [PID=6] 20th fib = 6765, bigarr[790]=2371
[Lab5 Fib] [CHLD] [PID=7] 20th fib = 6765, bigarr[790]=2371
[Lab5 Fib] [PARN] [PID=6] 25th fib = 75025, bigarr[748]=2245
[Lab5 Fib] [CHLD] [PID=7] 25th fib = 75025, bigarr[748]=2245
[Lab5 Fib] [PARN] [PID=6] 30th fib = 832040, bigarr[693]=2080
[Lab5 Fib] [CHLD] [PID=7] 30th fib = 832040, bigarr[693]=2080
[Lab5 Fib] [PARN] [PID=6] 35th fib = 9227465, bigarr[641]=1924
SHELL > [Lab5 Fib] [CHLD] [PID=7] 35th fib = 9227465, bigarr[641]=1924
run /fat32/mfork
[INFO, PID = 2] syscall.c:143:sys_clone: fork new process [pid = 8]

[Lab5 Mfork] Mfork Test Start
[INFO, PID = 8] syscall.c:143:sys_clone: fork new process [pid = 9]
```